

Mp10 Web — Installation

Installing and updating the browser application

Structured Systems · www.structuredsystems.com

10 August 2026

Contents

1	About this guide	3
2	Prerequisites	4
3	Before you install: verify the download	5
4	Installing	6
4.1	The six questions	6
4.2	The admin password	7
4.3	If the admin password was missed (<i>IT task</i>)	7
4.4	When it finishes	8
5	What it touches	9
5.1	The dedicated Apache instance	9
6	TLS certificate	10
6.1	Replacing the placeholder	10
7	Updating	11
8	Removing it	12
9	Fault-finding	13
9.1	Step 0/8 — Privileges	13
9.2	Step 1/8 — Verify the bundle	13
9.3	Step 2/8 — Preflight	13
9.4	Step 3/8 — Configuration	14
9.5	Step 4/8 — Test the dictionary connection	15
9.6	Step 5/8 — Deploy files	15
9.7	Step 7/8 — Apache (dedicated instance)	15
9.8	Step 8/8 — Database	16
9.9	Smoke test	17

1 About this guide

Mp10 Web is the browser-based front end for patients, encounters and insurance — a PHP 8 API and a Vue 3 single-page app, reading and writing the same Advantage Database Server dictionary (`mp.add`) the Mp10 Win32 desktop applications already use.

This guide is for whoever installs and maintains it on a server. It covers one thing: taking a published bundle (a single zip) and turning it into a running site with `install.ps1`, the script the bundle ships. Everything here is **IT task** — there is nothing in this guide for clinic staff.

If you are setting up a development copy from source instead of a customer site, see `Web/INSTALL.md` in the repository — the manual, step-by-step path remains there. This guide is the server-install path: one script, six questions, and a running site.

2 Prerequisites

The bundle carries the application only. PHP, Apache and the ADS client belong to the server, and the installer can only diagnose a wrong one, not fix it — so get these right first:

- **PHP 8.x, the ZTS (thread-safe) build, x64, with php_ads loaded.** Windows PHP ships four builds (NTS/ZTS $\times$ x86/x64); this needs the *Thread Safe* x64 zip from windows.php.net, with `extension=ads` enabled in `php.ini` and `php_ads.dll` in the extension directory.
- **The 64-bit ACE client on PATH** — `ace64.dll`, `adsloc64.dll`, `aicu64.dll`, `axcws64.dll`. **The Mp10 desktop applications install the 32-bit ACE client.** That is a different, separate install: it satisfies the desktop apps and will not satisfy this PHP. The installer’s preflight check exists specifically to catch this and says so by name.
- **php-cgi.exe next to php.exe.** Mp10 Web serves PHP through `mod_fcgid`, which needs the CGI build alongside the CLI one; a PHP zip normally ships both together, but confirm it.
- **Apache 2.4, a full build such as Apache Lounge’s**, with its standard module set on disk under `<Apache root>\modules\` — specifically `mod_rewrite.so`, `mod_fcgid.so`, `mod_ssl.so` and `mod_socache_shmcb.so`. Nothing needs *enabling* in that Apache’s own `httpd.conf` — see “What it touches” below for why — but the `.so` files themselves must exist on disk, because Mp10 Web’s own configuration loads them directly. A minimal or trimmed-down Apache build that omits `mod_fcgid` will not do.
- **openssl.exe in that Apache’s bin\ folder** (Apache Lounge ships it), *unless* you are supplying your own TLS certificate up front — see “TLS certificate” below.

3 Before you install: verify the download

The bundle publishes as two files: a zip (`mpweb-update-<version>.zip`) and a `.sha256` sidecar beside it. `manifest.json` inside the zip verifies every file *once extracted*, but it cannot verify itself, or the zip that carries it — that is what the sidecar is for, and it has to be checked **before** you extract, not after:

```
Get-FileHash .\mpweb-update-<version>.zip -Algorithm SHA256
```

Compare the `Hash` value against the contents of `mpweb-update-<version>.zip.sha256` (a plain text file: the hash, two spaces, the filename). If they do not match, download again — do not extract a zip whose checksum does not match its sidecar, even if it looks fine.

Once that checks out, extract the zip. `manifest.json` and `install.ps1` land beside each other at the top of the extracted folder — that is the “bundle” the rest of this guide refers to.

4 Installing

From an **elevated** PowerShell, in the extracted bundle folder:

```
.\install.ps1
```

Three optional parameters, if the defaults are wrong for this machine:

- `-InstallRoot <path>` — where Mp10 Web is deployed. Defaults to `C:\Mp10Web`.
- `-PhpPath <path to php.exe>` — which PHP to use. Defaults to whatever `php.exe` resolves to on PATH, or `C:\php\php.exe`.
- `-ApachePath <Apache root, or the bin\httpd.exe inside it>` — which Apache to borrow binaries and modules from. Either form is accepted.

Pass `-ApachePath` on any machine with more than one Apache. Without it the installer takes the first `httpd.exe` on PATH, which is whichever Apache happens to be registered — not necessarily the one Mp10 Web should use. It says so plainly when it falls back that way:

```
[WARN] Apache taken from PATH: c:\Apache24\bin\httpd.exe
[WARN] If that is not the Apache Mp10 Web should use, re-run with -ApachePath.
```

That warning is worth acting on. A machine can easily carry an Apache that serves something else entirely — through `mod_harbour`, or loading a different set of `php_ads` libraries — and Mp10 Web running on it will work until the day the other application’s needs and yours diverge.

Changing `-ApachePath` on a later run is safe and does the whole job: the installer notices the `Mp10Web` service is registered against the old binary, re-registers it against the new one, and restarts it. It tells you that is what it is doing.

The script runs eight steps plus a smoke test, in an order deliberately chosen so that **nothing is written until every check that can run has run** — a half-configured install is worse than a refused one:

Step	What happens
0/8 Privileges	Confirms you are elevated.
1/8 Verify the bundle	Hashes every file in <code>manifest.json</code> against what is actually on disk.
2/8 Preflight	Checks PHP’s version, bitness, thread-safety and the <code>ads</code> extension; checks Apache and its module files.
3/8 Configuration	Asks the six questions below.
4/8 Test the dictionary connection	Connects to <code>mp.add</code> with what you just typed, before writing anything .
5/8 Deploy files	Copies the application into the install root.
6/8 Configuration file	Writes <code>database.local.php</code> (fresh install only — see “Updating”).
7/8 Apache (dedicated instance)	Renders and validates a complete <code>httpd.conf</code> , generates a TLS certificate if needed, registers and starts the <code>Mp10Web</code> Windows service.
8/8 Database	Runs the migration: creates <code>web_users</code> , <code>web_groups</code> , <code>web_group_members</code> if they don’t exist, seeds the <code>admin</code> account, grants permissions.
Smoke test	Confirms an unauthenticated request to the running site is correctly refused.

4.1 The six questions

Asked once, interactively (skipped in favour of defaults or a saved answer under `-NonInteractive`):

1. **ADS mode — Remote or Local.** Remote is normal: the dictionary lives on an ADS server elsewhere on the network.
2. **Path to mp.add.** The existing dictionary — remote paths look like `\\HOST:6262\VOLUME\path\mp.add`.
3. **ADS username.** Defaults to `adssys`.
4. **ADS password** (typed hidden).
5. **Port for Mp10 Web.** Suggests 8443, or the next free port above it if that one is taken.
6. **Public origin.** The exact address staff will type into their browser — sets both `cors_origins` (in `database.local.php`) and the certificate's hostname. Defaults to `https://<this machine's name>:<the port above>`.

4.2 The admin password

On a **first** install, step 8/8 creates the **admin** account and prints a randomly generated password to the terminal — once:

```
>>> ADMIN PASSWORD: <generated>
Recorded nowhere else. Write it down now.
```

That is not an exaggeration. The password is bcrypt-hashed into `web_users` immediately; nothing on the server keeps the plain text anywhere, and there is no “forgot password” flow — only a reset. If it scrolls past before you copy it, the account is not lost, but recovering access means resetting it, not retrieving it.

The installer writes no log, deliberately — a credential sitting in a plaintext install log is worse than one you have to write down. So the terminal really is the only place it appears.

4.3 If the admin password was missed (*IT task*)

Re-running the installer does **not** help: the migration is idempotent, so once the **admin** account exists it reports **already present** and never prints a password again.

Delete the account and let the migration recreate it. Two statements against the dictionary — Advantage Data Architect, or any SQL tool connected to `mp.add`:

```
DELETE FROM web_group_members
WHERE user_id IN ( SELECT user_id FROM web_users WHERE username = 'admin' );

DELETE FROM web_users WHERE username = 'admin';
```

Both statements, in that order. `web_users.user_id` is an AUTOINC column, so the recreated account gets a *new* id — deleting only the `web_users` row leaves its group membership behind, pointing at a user that no longer exists.

Then re-run the migration on the server:

```
C:\php\php.exe C:\Mp10Web\tools\migrate.php
```

It recreates the account and prints a fresh password the same way the installer did:

```
Create the admin group and user.....created admin / <generated> - record this
now, it will never be shown again. Change it on first login.
```

Nothing else is disturbed. The **Administrators** group and its permissions are left exactly as they were, and **any other web users, and their group memberships, are untouched** — only the **admin** account is recreated. No service restart is needed.

This procedure works for any web account, not just **admin** — but the migration only ever recreates **admin**. Deleting anyone else simply removes them.

4.4 When it finishes

A successful run ends with a summary:

Mp10 Web is installed.

```
URL           : https://<host>:<port>
Install root  : C:\Mp10Web
Config        : C:\Mp10Web\backend\php\config\database.local.php
Logs          : C:\Mp10Web\logs
Apache conf   : C:\Mp10Web\apache\conf\httpd.conf
Apache svc    : Mp10Web
TLS cert      : C:\Mp10Web\apache\conf\server.pem
```

Open the URL. If the certificate is the self-signed placeholder (see below), the browser will warn — that is expected until it is replaced.

5 What it touches

Deliberately little, and every bit of it is listed here:

- **Three tables in the dictionary:** `web_users`, `web_groups`, `web_group_members`. The clinical schema, its stored procedures, and the `NewSeqKey` key-minting function all belong to the **Win32 Admin module** — this installer neither creates nor inspects them. If a dictionary is missing something Mp10 Web needs at that level, that is fixed by running Admin, not by this installer.
- **One row in the existing sequences table**, `field = 'recno'`. Every install and update reads the current highest `patients.recno` and raises `sequences`'s `recno` counter to match, if it is not already there — so a record number Mp10 Web mints can never collide with one that already exists. It only ever raises the value, never lowers it, and does nothing on a run where the counter is already caught up. This is a write to dictionary infrastructure the desktop apps also read, not a Mp10-Web-owned table — see “Removing it” below for why it must not be deleted.
- **One config file**, `database.local.php`, under the install root.
- **One rendered Apache configuration and one certificate/key pair**, both under the install root — never under the system Apache's own configuration directory.
- **One Windows service**, named `Mp10Web`.

5.1 The dedicated Apache instance

Mp10 Web does **not** add a virtual host or an `Include` line to the server's existing Apache. It runs its **own**, complete, standalone Apache instance:

- A full `httpd.conf` is rendered under `<install root>\apache\conf\` — its own explicit `LoadModule` list, its own `Listen` port, its own logs, its own service (`Mp10Web`).
- `ServerRoot` in that rendered config points at the **existing** Apache installation, purely so `modules/mod_*.so` and `conf/mime.types` resolve — nothing under the existing installation is ever written to, and its own `httpd.conf` and its own service are never read, edited, or restarted.
- PHP is served through `mod_fcgid`, wired to the server's **existing** `php-cgi.exe` and `php.ini` — the same PHP the preflight step already validated. The bundle ships no PHP of its own.
- TLS is terminated on this instance (see below) — it is not layered behind the system Apache or any other reverse proxy.

Because the system Apache's configuration and service are never touched, there is nothing to undo on it when Mp10 Web is removed, and nothing on it that a later Mp10 Web update can break.

6 TLS certificate

The dedicated instance always serves over HTTPS. On a fresh install, if no certificate is already present, `install.ps1` generates a **self-signed placeholder** using the existing Apache's `openssl.exe`, and says so loudly:

```
[ .. ] No certificate found at '...\apache\conf\server.pem' - generating a self-signed placeholder
[ OK ] Generated self-signed certificate: CN=<host>
[WARN] SELF-SIGNED placeholder installed. Replace ...\server.pem and
[WARN] ...\server.key with a real certificate, then restart the 'Mp10Web' service.
```

Browsers will show a certificate warning until this is replaced. That is expected for the placeholder and is not itself a fault.

6.1 Replacing the placeholder

Put a real certificate and private key at:

```
<install root>\apache\conf\server.pem
<install root>\apache\conf\server.key
```

then restart the `Mp10Web` service. If the certificate lives somewhere else instead, hand-edit the rendered `httpd.conf`'s `SSLCertificateFile` / `SSLCertificateKeyFile` lines to point at it — a later update honours whatever the *live* config already points at, so the edit survives.

Either way, once a real certificate is in place, `install.ps1` will never regenerate or overwrite it on a later update — the same rule that protects `database.local.php`'s secrets applies here.

7 Updating

Run the same command against the same install root:

```
.\install.ps1
```

(-InstallRoot only needs to be given again if you did not install to the default C:\Mp10Web in the first place.)

The six questions come back — pre-filled with the answers already on file. Step 3/8 reads the existing `database.local.php` and offers its values (dictionary path, ADS username and password, port, public origin) as the default for each prompt; pressing Enter through all six keeps the site exactly as it is. This is also what makes an unattended `-NonInteractive` update work at all — with no console to prompt at, every answer falls back to what was already on file, so nothing needs to be typed for a routine update.

What is kept, deliberately, beyond that:

- **database.local.php** — never rewritten once it exists. It holds the generated JWT secret; regenerating it would sign out every currently logged-in user.
- **The TLS certificate** — never regenerated once real files exist at the resolved cert/key paths. Same fingerprint before and after.
- **The Mp10Web service** — restarted only if the rendered Apache configuration actually changed; left alone otherwise.

What always re-runs: **the migration** (step 8/8). It is idempotent — a second run reports what was already there (**already present, already granted**) rather than doing it again — but it is not optional. **New permissions arrive only through the migration.** Permissions are stored as data in `web_groups.perms`, with no admin screen to grant them yet, so a feature gated on a permission that shipped in this update is simply invisible — no error, no empty state — until the migration that grants it actually runs. Skipping the update-and-rerun step because “nothing looked different” is exactly how that gap gets missed.

8 Removing it

There is no `httpd.conf` `Include` line to un-edit — the dedicated instance is self-contained, so removal is:

```
<path to existing Apache>\bin\httpd.exe -k stop -n Mp10Web  
<path to existing Apache>\bin\httpd.exe -k uninstall -n Mp10Web
```

then drop the three tables (`web_users`, `web_groups`, `web_group_members`) from the dictionary, and delete the install root. The system Apache was never touched, so nothing there needs reverting.

Do not delete the sequences row for field = 'recno'. It is not Mp10-Web-owned data — it is shared dictionary infrastructure the desktop apps read too, and removing it (rather than merely leaving Mp10 Web's own three tables behind) would affect record-number minting outside this application entirely. Removing Mp10 Web means dropping the three `web_*` tables; it does not mean touching `sequences`.

9 Fault-finding

Every failure the installer can produce is printed with **[FAIL]** and stops the run at that point. **Before step 5/8 (Deploy files)**, that means nothing was written — whatever was there before, if anything, is exactly as it was. **From step 5/8 onward**, though, the application files under the install root have already been replaced by the time a later step can fail — a failure past that point leaves a partially-updated tree, not a rolled-back one. Treat any such failure as **incomplete, not safe**: fix the problem and re-run to a clean completion before trusting the site again. Apache’s own rendered configuration is the one deliberate exception even within that range — it is always written to a temporary file and validated with `httpd -t` before it is ever moved into the path the running service actually reads (step 7/8), so a rejected render never touches what is currently live. The table below is keyed to the exact message text; where a message embeds a value (a path, a count, a port), that part is shown as `<...>`.

9.1 Step 0/8 — Privileges

Message	What it means	What to do
Run this from an elevated PowerShell. It writes to the Apache configuration and the install root.	PowerShell was not run as Administrator.	Re-open PowerShell with “Run as administrator” and re-run.

9.2 Step 1/8 — Verify the bundle

Message	What it means	What to do
No manifest.json beside install.ps1 - is this an extracted Mp10 Web bundle?	<code>install.ps1</code> was run from somewhere other than the extracted bundle.	Run it from inside the extracted zip, next to <code>manifest.json</code> .
<code><n></code> file(s) missing or corrupt. Re-copy the bundle and extract it again.	Extraction was incomplete, or a file changed after extracting.	Delete the extracted folder and re-extract from a zip you have already checked against its <code>.sha256</code> sidecar.
manifest.json claims <code><N></code> files but lists <code><M></code> - the manifest itself looks corrupt. Re-copy the bundle and extract it again.	<code>manifest.json</code> itself is damaged.	Re-download the zip (checksum it first) and re-extract.
This installer build handles the ‘update’ flavour; this bundle says ‘ <code><flavour></code> ’.	The extracted bundle is not an <code>update</code> -flavour bundle.	Use the bundle that matches this installer; do not mix bundle types.

9.3 Step 2/8 — Preflight

Message	What it means	What to do
PHP not found at ‘ <code><path></code> ’. Install PHP 8.x ZTS x64, or pass <code>-PhpPath <path to php.exe></code> .	No PHP at the default location or on <code>PATH</code> .	Install PHP 8.x ZTS x64, or pass <code>-PhpPath</code> .
The PHP probe produced no output. ‘ <code><path></code> ’ may not be a working PHP: <code><raw></code>	The file at that path is not a functioning <code>php.exe</code> .	Confirm the path is really <code>php.exe</code> ; the quoted <code><raw></code> text is whatever it actually printed.

Message	What it means	What to do
PHP <version> found; Mp10 Web needs PHP 8.x. Install an 8.x build and re-run.	PHP is the wrong major version.	Install PHP 8.
PHP is <bits>-bit; Mp10 Web needs the x64 build. Note the Mp10 DESKTOP applications are 32-bit - this is a different PHP, not the same one.	The 32-bit trap. A 32-bit PHP was found — commonly because that is what happens to be on the machine already.	Install a separate PHP 8 x64 build for Mp10 Web; do not point it at whatever 32-bit PHP is already there.
PHP is the NTS (non-thread-safe) build; Mp10 Web's php_ads.dll needs the ZTS build. Download the 'Thread Safe' x64 zip from windows.php.net.	Wrong PHP thread-safety variant.	Install the <i>Thread Safe</i> x64 zip, not the <i>Non Thread Safe</i> one.
The Advantage PHP extension is not loaded - AdsConnection does not exist. Copy php_ads.dll into <ext_dir> and add extension=ads to <ini>. It also needs the 64-BIT ACE client (ace64.dll, adsloc64.dll, aicu64.dll, axcws64.dll) on PATH - the Mp10 desktop applications install the 32-bit client, which will NOT satisfy this.	php_ads is missing, disabled, or the 64-bit ACE client isn't on PATH.	Enable extension=ads in the named php.ini with php_ads.dll in the named extension directory, and put the 64-bit ACE DLLs on PATH — not the desktop's 32-bit ones.
No Apache found. Pass -ApachePath <Apache root or bin\httpd.exe>, or install Apache 2.4 (an Apache Lounge build ships every module needed) and re-run.	No Apache on PATH, none shipped in the bundle, and none at C:\Apache24.	Pass -ApachePath if Apache is installed somewhere else; otherwise install Apache 2.4.
-ApachePath '<path>' does not lead to bin\httpd.exe (looked for '<resolved>').	The path given is neither an Apache root nor an httpd.exe.	Give either the Apache root (the folder containing bin\ and modules\) or the full path to bin\httpd.exe.
Apache at '<modules_dir>' is missing: <mod_rewrite.so, mod_fcgid.so>. Mp10 Web's own Apache instance (step 7/8) needs mod_rewrite (the API's .htaccess re-attaches the Authorization header Apache strips from the CGI environment - without it every signed-in request 401s WITH a valid token) and mod_fcgid (runs PHP). Install a full Apache 2.4 build that ships both modules.	The found Apache's modules\ folder is missing one or both files.	Install a full Apache 2.4 build (e.g. Apache Lounge) that ships mod_rewrite.so and mod_fcgid.so — a trimmed-down build will not do.

9.4 Step 3/8 — Configuration

Message	What it means	What to do
A dictionary path is required.	The “Path to mp.add” prompt was left blank.	Supply the real path to <code>mp.add</code> .
An ADS username is required.	The username prompt was left blank.	Supply an ADS username.
ADS password is required and this session cannot prompt for one (-NonInteractive, or no console, with no existing install to reuse). Re-run interactively, or install once interactively first.	Running unattended with no saved password to fall back on.	Install once interactively so a password is on file, or run interactively this time.
Port <code><port></code> is in use by another process (not the install being updated). Choose another.	The chosen port is held by something other than this same install.	Pick a different port at the prompt.

9.5 Step 4/8 — Test the dictionary connection

Message	What it means	What to do
The connection test produced no result. Check that <code>php_ads</code> is loadable.	PHP failed to run the connection probe at all.	Run <code>php -m</code> and confirm <code>ads</code> loads without error.
Could not open the dictionary: <code><ADS error></code>	The path, credentials, or network path to <code>mp.add</code> is wrong, or the server is unreachable.	Read the quoted ADS error — it states exactly which of those it was. Password is redacted from this message even on a credential failure.

9.6 Step 5/8 — Deploy files

Message	What it means	What to do
robocopy failed copying ‘ <code><dir></code> ’ to <code><root></code> (exit code <code><code></code>). Check permissions on the destination.	The install root could not be written to (permissions, a locked file, antivirus).	Check NTFS permissions on the install root and that nothing has a file there open/locked. Directories copied earlier in this same step may have already succeeded before this one failed, so the install root is now a MIXED, incomplete tree — do not point Apache at it or consider the site trustworthy until you fix the problem and re-run to a clean completion.

Step 6/8 (Configuration file) has no row here on purpose: writing a fresh `database.local.php`, or keeping an existing one, has no distinct failure message to catch — nothing was skipped.

9.7 Step 7/8 — Apache (dedicated instance)

Message	What it means	What to do
php-cgi.exe not found next to ' <code><PhpPath></code> ' (looked for ' <code><phpCgi></code> '). Mp10 Web serves PHP through <code>mod_fcgid</code> , which needs the CGI build alongside the CLI one you pointed <code>-PhpPath</code> at.	<code>php-cgi.exe</code> is missing beside <code>php.exe</code> .	Install/copy <code>php-cgi.exe</code> into the same folder as the <code>php.exe</code> being used.
Public origin ' <code><origin></code> ' is not a valid host[:port] or URL.	The “Public origin” answer could not be parsed at all.	Re-enter it as <code>host</code> , <code>host:port</code> , or a full <code>https://host:port</code> URL.
Could not derive a hostname from public origin ' <code><origin></code> '.	The answer parsed but produced no host.	Same fix as above.
<code>openssl.exe</code> not found at ' <code><path></code> '. Mp10 Web needs it once, to generate a self-signed placeholder certificate. Either install an Apache build that ships OpenSSL, or place a real certificate yourself at ' <code><pem></code> ' / ' <code><key></code> ' before re-running.	No certificate exists yet, and the existing Apache has no <code>openssl.exe</code> to generate a placeholder.	Use an Apache Lounge build (ships OpenSSL), or supply your own certificate/key at the named paths before re-running.
Certificate generation failed - see <code><log></code>	<code>openssl req</code> itself failed.	Read the tail of the named log file for the OpenSSL error.
Apache rejected the configuration - see above. <code><conf></code> was left untouched, but the application files under <code><root></code> were already updated by step 5/8 of this run. This update is INCOMPLETE - fix the problem above and re-run to a clean completion before trusting this site.	<code>httpd -t</code> found a problem in the rendered config — most commonly a missing module file (<code>mod_ssl.so</code> , <code>mod_socache_shmcb.so</code> , etc.) that preflight does not check by name.	Read the <code>httpd -t</code> output printed just above this line; confirm every module file the template loads exists under Apache's <code>modules\</code> . The Apache config itself is unchanged, but do NOT treat this run as complete — the application files were already deployed — until you fix the problem and re-run to a clean finish.
Could not register the ' <code><serviceName></code> ' service - see above.	<code>httpd -k install</code> failed — commonly a stale service left in a bad state, or a permissions issue.	Read the output above; confirm elevation, and check for a leftover Mp10Web service (<code>Get-Service Mp10Web</code>) that needs removing first.
Could not start/restart ' <code><serviceName></code> ': <code><error></code>	The service was registered but would not start.	The install log's last 20 lines from <code><root>\logs\mpweb-error.log</code> print automatically above this message — read them.

9.8 Step 8/8 — Database

Message	What it means	What to do
The database step failed - see above.	<code>tools\migrate.php</code> exited with an error, or printed a real (uppercase) <code>FAILED</code> line.	Read the migration output printed above — it names the specific step that failed.

Message	What it means	What to do
WARNING: sequence=<n>, but the next number (<recno>) is already taken...	<code>sequences.recno</code> has fallen behind the patient record numbers actually in use. Patient creation still works — it probes for a free number — but it is retrying past taken ones, and enough of them in a row will exhaust its 50 attempts.	Investigate in Admin10 before creating patients. Do not “fix” it by raising the counter blind: that row is shared with the desktop applications, which mint patient numbers from it too, and <code>MAX(recno)</code> is not a reliable target — one mistyped record number sitting above the real block would burn every number in between, irreversibly.
The admin password scrolled past / was never seen	The account exists; the password does not exist anywhere.	See If the admin password was missed , above.

9.9 Smoke test

Message	What it means	What to do
GET /api/auth/me returned <code>; 401 was expected. Check <root>\logs.	The site answered, but not the way a healthy, unauthenticated request should.	Check <root>\logs\mpweb-error.log and the API's own logs for what happened on that request.
GET /api/auth/me returned <code> without a token; 401 was expected.	An unauthenticated request was not refused — treat this as serious, not cosmetic.	Do not consider the site ready until this passes; check the logs as above.
Could not reach <base>/api/auth/me after 20s (connection refused/not listening): <err>. Check the 'service' service and <root>\logs.	Nothing answered the configured port within 20 seconds of the service starting.	Check Get-Service Mp10Web and <root>\logs\mpweb-error.log.